

从会用工具，到能判断 Agent 架构好坏

目标不是逐行啃源码，而是建立工程判断力：看懂 agent loop、状态机、上下文管理、MCP 与 Skill 发现、工具调用、记忆管理，并用一个全栈项目落地。

3 层

扫盲、架构理解、项目落地

5 个

重点项目与教程资源

1 个

可写进简历的垂直 Agent 项目

先建立判断力，再追求代码量

不要一上来手搓框架

先理解优秀 agent harness 中模型、工具、记忆、权限、状态与上下文如何协同。

不要只做聊天框

能落地的项目要有工具约束、评测集、失败分析和成本控制。

善用 AI 编程

用 Codex、Claude Code 或同类 coding plan 做方案讨论、文档初始化、生成与迭代。

路线总览：从概念扫盲到工程落地

时间有限时，把源码阅读当作辅助，把「能否解释系统为什么这样设计」作为主线。

阶段 01 入门扫盲：知道 Agent 领域在讨论什么

用 hello-agents 补齐概念。重点不是抄代码，而是建立词汇表：Agent、Tool、Memory、RAG、Multi-Agent、Workflow、Evaluation。

输出一页概念地图：每个概念对应哪个工程模块。

先读懂 README、架构图和模块命名，再追源码细节。

阶段 02 深入理解：看懂 agent loop 与 harness

看 learn-claude-code 与 learn-harness-engineering。关注状态机、上下文裁剪、工具协议、权限边界、错误恢复、记忆、Skill 与 MCP 发现。

画出用户输入、工具执行、模型续写、结果落库的完整链路。

说明哪些环节用代码，哪些交给模型，哪些需要人类确认。

阶段 03 项目拆解：用 Craft Agents 建立全栈参照系

Craft Agents OSS 是全栈 Agent 参考：类型层、共享业务逻辑、桌面端、服务端、CLI、WebUI、会话查看器分层清晰。

先用 Zread 读架构报告，不必从源码第一行硬啃。

用 Codex 创建项目理解 Skill，作为后续系统设计参考。

阶段 04 方案设计：用 Spec 与 Plan 模式收敛项目

开 Plan 模式明确业务场景、用户路径、工具边界、数据源、权限策略、失败处理和评测方式。

先定义行为，再让 coding agent 实现。

让 AI 初始化 PRD、架构、信源、编码规范和迭代清单。

阶段 05 垂直落地：做一个能演示、能评测、能优化的 Agent

从垂直场景切入：股票、客服、标书、质检、运维、翻译或数据分析。先做窄，再做深。

工具收窄到业务动作，例如 K 线、市值、PE、持仓、预警。

评测集、工具说明、消融实验和成本控制，是区分玩具与产品的核心。

推荐项目：分别训练不同能力

每个项目只抓一个目标。先把它们当能力训练材料，不要都当源码题刷。

hello-agents

入门扫盲项目。快速了解 Agent 概念、教程脉络和常见场景，重点是术语和问题意识。

[打开 GitHub](#)

[概念扫盲](#) [Agent 基础](#) [适合第一站](#)

learn-claude-code

用小型实现理解 Claude Code 类 harness。重点看 agent loop、工具调用、上下文、状态推进和错误处理。

[打开 GitHub](#)

[agent loop](#) [状态机](#) [tool calling](#)

learn-harness-engineering

学 harness engineering：如何设计输入输出、工具边界、观测、回放、评测和失败恢复。

[打开 GitHub](#)

[harness](#) [工程边界](#) [评测思维](#)

craft-agents-oss

全栈 Agent 参照。学习类型层、共享业务逻辑、Claude/Pi 后端、来源管理、会话、权限、自动化如何组合。

[打开 GitHub](#)

[全栈 Agent](#) [Electron](#) [MCP 与 Skill](#) [架构参考](#)

Zread 项目解析

把以上项目丢给 Zread 做结构化解析。先读概述、架构、模块边界和关键流程，再决定是否深入源码。

[打开 Zread](#)

[仓库解析](#) [快速建立全局视角](#) [减少无效啃源码](#)

Craft Agents 值得学习的设计点

它适合作为全栈 Agent 架构样板：不必复刻，但要能解释为什么这样分层。

类型层与业务层分离

`@craft-agent/core` 放共享类型；`@craft-agent/shared` 放核心业务；应用层只消费契约。

多后端 Agent 抽象

Claude 与 Pi 后端并行，应用不绑定单一 SDK，方便做多提供商路由与成本分层。

来源、权限与自动化

MCP、REST API、本地文件、会话工具、权限模式和自动化进入统一运行时，而不是散落在 UI 里。

技术栈参考

参考这套组合即可。重点不是堆技术，而是让每层职责清晰。

层级	技术	学习重点
运行时	Bun, TypeScript strict ESM	统一包管理、脚本、类型约束和构建入口。
桌面框架	Electron, esbuild	主进程、预加载脚本、渲染器和本地权限边界。
UI 层	React, Tailwind CSS, Jotai, Radix UI	会话列表、消息流、工具调用态、设置面板和响应式交互。
Agent SDK	Claude Agent SDK, Pi Coding Agent	后端抽象、流式事件、工具调用和权限确认。
MCP	@modelcontextprotocol/sdk	工具发现、外部服务接入、协议边界和可扩展来源。
传输层	JSONL WebSocket RPC	流式事件、会话恢复、远程服务端和瘦客户端模式。
校验与观测	Zod, Sentry	输入输出 schema、错误分类、日志、回放和线上问题定位。

第一个项目怎么做

做一个能演示、能解释架构、能展示迭代过程的项目。小也可以，但要有业务闭环。

1. 准备 AI 编程环境

至少开一个 coding plan。核心是能持续 Plan、执行、验证、复盘。

2. 建项目理解 Skill

让 Codex 理解 Craft Agents OSS，再沉淀成架构参考 Skill。

3. 选 Agent 框架

试 Pi Agent SDK 或 Claude Agent SDK。把文档和示例仓库变成开发 Skill。

4. 先写 Spec，再 vibe coding

先定 PRD、架构、数据源、工具契约、评测计划和错误处理，再生成系统雏形。

Agent 场景细化清单

雏形之后，真正拉开差距的是业务收窄、工具设计、评测和优化。

业务与工具

- ✓ 明确目标用户和高频任务，不做泛泛的万能助手。
- ✓ 工具收窄到业务函数：K 线、市值、PE、持仓、新闻、预警。
- ✓ 写清参数、返回值、失败情况、成本和禁止场景。
- ✓ 把 Skill 做成可维护资产，而不是一次性提示词。

评测与优化

- ✓ 建立小而准的评测集，覆盖正常、边界和诱导错误。
- ✓ 记录失败归因：工具、上下文、模型、业务规则。
- ✓ 跑消融实验，判断哪些机制有正贡献。
- ✓ 对比不同模型、提示词、工具粒度下的成功率、时延和成本。

如何把项目写进简历

不要只写「基于大模型实现智能问答」。要写架构、工具、评测和优化。

架构表达

写 agent loop、工具注册、会话状态、上下文裁剪、权限确认、记忆和错误恢复。

业务表达

写业务场景、关键工具、数据源、用户路径和约束条件。

结果表达

写评测集规模、成功率、失败类型、成本优化和消融实验结论。

最终目标：不是背框架，而是形成判断力：看懂核心循环，解释工具发现、上下文管理、权限控制、失败处理和持续优化。再结合目标岗位 JD 做一个垂直项目，就能得到可展示、可复盘、可写进简历的作品。

建议阅读顺序：概念扫盲 → agent harness → Craft Agents 全栈架构 → Spec/Plan 开发 → 垂直场景评测优化